Kong - Estratégia de migração

Last updated by | Felipe Molinari | 27 de mar. de 2026 at 15:35 BRT

Migração Sensedia → Kong: Levantamento Técnico

Contexto

A Pottencial utiliza o **Sensedia API Gateway** como camada de roteamento. A migração para o **Kong Gateway** precisa preservar o comportamento das rotas existentes — incluindo parâmetros de path sensíveis como `product_key`, reescrita de URLs e injeção automática de headers.
A estratégia central foi: **não migrar manualmente rota por rota**, mas criar um script utilizando a **API Admin do Kong** para automatizar esse trabalho.

Estratégia de Migração

Para habilitar a migraçao, o time criou um arquivo .xlsx contendo todos os serviços, rotas do gateway e o roteamento para os serviços upstream.
O script `kong-importer-excel.js` lê esse Excel e provisiona os recursos diretamente no Kong via Admin API.

1. Excel como fonte da verdade

Coluna	Descrição
`Gateway Endpoint`	Path público exposto no gateway (ex: `/insurance/v2/{product_key}/quotes/{quote_id}`)
`Backend Endpoint`	Path real do serviço backend (ex: `/quotes/v2/quotes/:quoteId`)
`Method`	Método HTTP
`BackendController`	Nome do serviço/controlador backend

Colunas opcionais mapeadas:

Coluna	Finalidade
`Custom Headers`	Headers fixos a injetar no upstream
`Http Method`	Override do método HTTP
`Insert Product-key into payload`	Adiciona `productKey` no body da requisição
`Ignore`	Pula a rota na importação
`Route Name`	Nome customizado da rota no Kong

---

2. Rotas com Expressions (Kong Router Flavor)

O Kong foi configurado com `KONG_ROUTER_FLAVOR=expressions`, que permite usar regex nomeado no roteamento.

Por que isso foi necessário?

O Sensedia capturava parâmetros de path (como `{product_key}`) e os redirecionava. O Kong padrão não oferece esse nível de flexibilidade sem expressions.

Conversão automática:

```
Gateway path:  /insurance/v2/{product_key}/quotes/{quote_id}

Expression gerada:
  (http.method == "GET") &&
  (http.path ~ "^/insurance/v2/(?<product_key>[^/]+)/quotes/(?<quote_id>[^/]+)$")
```

Rotas **sem parâmetros** continuam usando o formato simples (`paths: ["/insurance/quotes"]`).

---

3. Reescrita de Path (request-transformer)

Quando o path do gateway difere do path do backend, o plugin `request-transformer` é aplicado automaticamente na rota.

Regra de mapeamento de parâmetros — simples por posição (1:1):

- `{product_key}` → sempre vai para o **header** `x-product-key` (nunca para o path do backend, a menos que o backend também tenha um `:productKey`)
- Demais parâmetros → mapeados por **ordem de aparição** (não por nome)
  Exemplo:

  Gateway: /insurance/v2/{product_key}/quotes/{quote_id}/items/{item_id}
  Backend: /quotes/v2/quotes/:quoteId/items/:itemId

  Mapeamento:
  {product_key} → header x-product-key (sempre)
  {quote_id} → :quoteId (1º parâmetro do backend)
  {item_id} → :itemId (2º parâmetro do backend)

  Plugin request-transformer gerado:
  replace.uri: /quotes/v2/quotes/$(uri_captures.quote_id)/items/$(uri_captures.item_id)
  add.headers: x-product-key:$(uri_captures.product_key)

> O nome dos parâmetros não precisa ser igual entre gateway e backend. O mapeamento é por posição.

Caso especial — paths compatíveis: quando o número e estrutura dos parâmetros são idênticos entre gateway e backend, o `replace.uri` **não é criado** (evita plugin desnecessário).

---

4. Plugin OAuth Client Credentials (Hefesto) - **Esse plugin precisa de refinamento, a versão feita foi para habilitar os testes da ferramenta**

Para que o Kong autentique no upstream (serviços internos protegidos pelo Hefesto), foi criado um **plugin Lua customizado** `oauth-client-credentials`.

Funcionamento:

1. Requisição chega no Kong
2. Plugin verifica cache de token (TTL ~55 min)

3. Se não há token válido: chama `POST /connect/token` no Hefesto com `grant_type=client_credentials`
4. Injeta `Authorization: Bearer {token}` na requisição antes de encaminhar ao upstream
   Suporte a **múltiplos ambientes** (dev / hml / prod) via header ou query param na requisição.

---

5. Plugins Customizados (Lua)

Três plugins foram desenvolvidos e montados via volume Docker:

Plugin	Finalidade
`oauth-client-credentials`	Busca e injeta token Bearer do Hefesto automaticamente
`custom-headers`	Injeta headers fixos: `x-person-id`, `x-product-key`, `x-person-role` - **Não será utilizado na versão final. Foi criado para habilitar os testes, já que as upstreams api's exigem esses campos no header para correto funcionamento**
`trace-id`	Copia `x-kong-requestid` para `x-trace-id` (rastreabilidade distribuída)

Dificuldades e Impedimentos

1. Autenticação de Consumidores (OAuth 2.0 — Client Credentials)

Situação atual (Sensedia):

Os consumidores fazem uma requisição `POST /oauth/v3/access-token` com Basic Auth (`client_id` / `client_secret`) e `grant_type=client_credentials` na API Gateway, o token retornado é usado nas chamadas subsequentes às APIs.

Problema identificado no Kong Konnect:

O plugin nativo `oauth2` **não está disponível na versão Konnect** (conforme documentação oficial da Kong). Esse era o plugin natural para expor um endpoint de geração de token e autenticar consumidores no gateway.

Impacto: O fluxo de autenticação dos consumidores não pode ser replicado diretamente. Caso o plugin OAuth2 realmente não seja disponível na versão konnect, será necessário avaliar alternativas.

Severidade: Alta — bloqueia a migração sem uma solução alternativa definida.

---

2. Extra Fields por Consumidor (Metadados Customizados)

Situação atual (Sensedia):

Cada consumidor possui propriedades adicionais configuradas no gateway (ex: flags de acesso, identificadores internos). Esses campos são lidos pelas aplicações backend para controlar acesso a determinados recursos.

Problema no Kong Konnect:

A entidade `Consumer` do Kong não suporta campos customizados arbitrários. O campo `custom_id` existe, mas é apenas uma string simples — não comporta estruturas de dados complexas ou múltiplos atributos.

Impacto: As aplicações que dependem desses metadados, caso eles não sejam informados os serviços não irão funcionar.

Severidade: Alta — afeta o funcionamento das APIs internas sem mudança de arquitetura.

---

3. Controle de Acesso por Consumidor (ACL por API)

Situação atual (Sensedia):

É possível restringir um consumidor a acessar apenas uma API específica — outros serviços são bloqueados para aquele consumidor.
Ainda não tivemos contato com essa funcionalidade do kong mas pelo que foi informado, esse comportamento é replicável no Kong Konnect.

Severidade: Média — Funcionalidade exigida

---

4. Proteção contra SQL Injection e XSS

Situação atual (Sensedia):

Todos os serviços possuem interceptores de proteção contra SQL Injection e XSS Injection aplicados globalmente.

Severidade: Alta — requisito de segurança presente em todos os serviços atualmente.

---

5. Observabilidade: Logs, OpenTelemetry e Datadog

Situação atual:

A Pottencial utiliza **Datadog** como plataforma de observabilidade, integrada via **OpenTelemetry**.

O que foi verificado no Kong:

- O Kong possui plugin nativo `opentelemetry` que exporta traces no formato OTLP
- O Datadog aceita dados OTLP via seu agent (porta `4317` gRPC ou `4318` HTTP)
- A integração é tecnicamente viável, mas **não foi testada na POC**

Pontos a validar:

- Configuração do `opentelemetry` plugin apontando para o Datadog Agent
- Correlação de `trace_id` entre Kong e as aplicações internas (uso do plugin `trace-id` desenvolvido na POC)
- Logs estruturados via plugin `file-log` ou `http-log` para indexação no Datadog
- Compatibilidade de atributos de span com os dashboards existentes

Severidade: Média — solução existe no Kong, mas requer validação e configuração específica para o ambiente Pottencial.